



ค่ายโอลิมปิกวิชาการ
ศูนย์มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

Problem Solving

สทิษฐ์ ประสมพันธ์

Sathit.p@sci.kmutnb.ac.th



Problem Solving Paradigms

Tree Basics

- Types of Trees
- Tree Traversal
- Binary Search Tree (BST) Operations
- AVL Tree (Balanced Binary Search Tree)

Graph Algorithms

- Graph Representation
- Graph Traversal
- Shortest Path Algorithms
- Minimum Spanning Tree (MST)

Brute Force

- Iterative Complete Search Recursive Complete Search

Divide and Conquer

- Binary Search
- การแปลง Optimization Problem เป็น Decision Problem

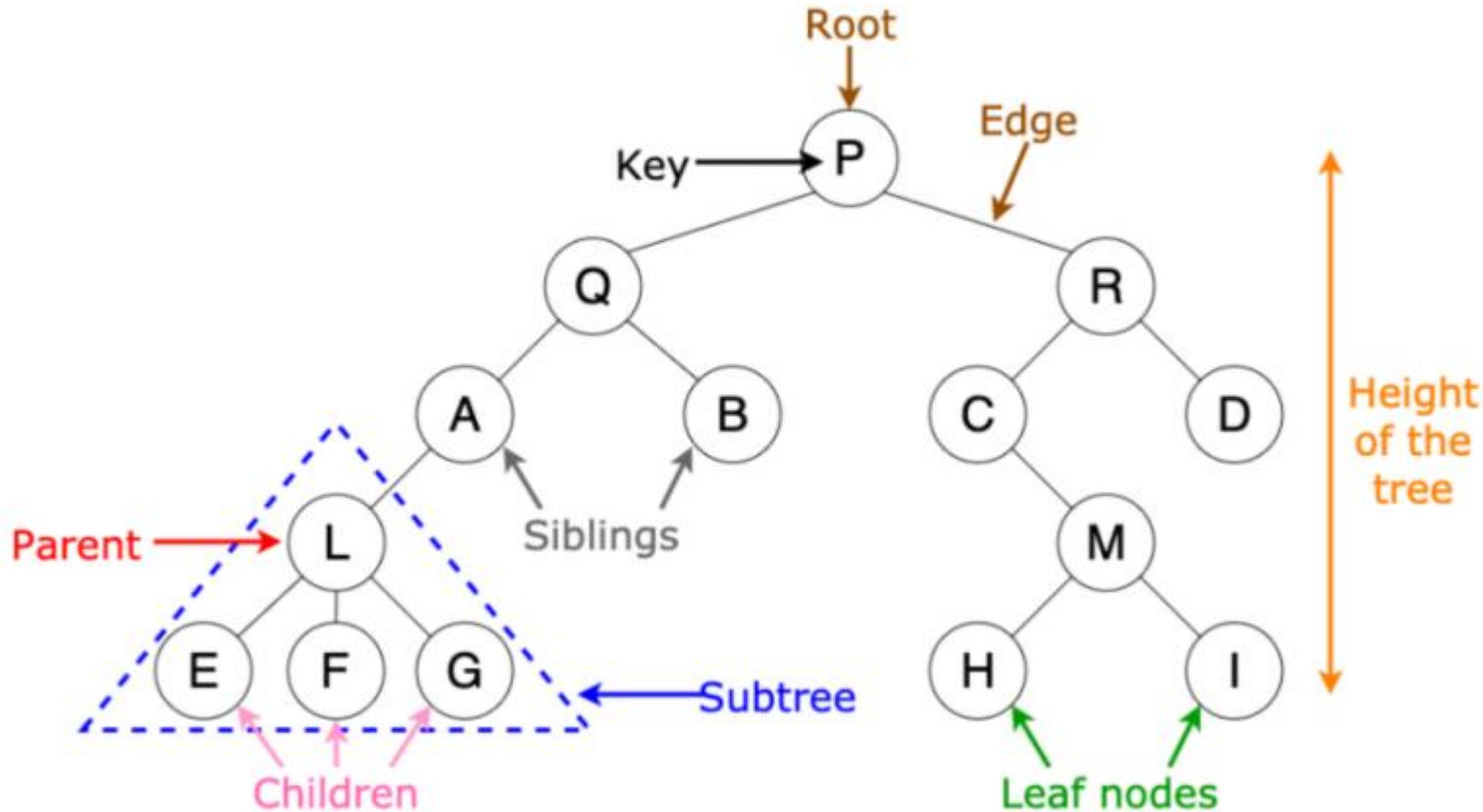
Greedy

Problem Solving with Backtracking



Tree

Tree Basics



A **tree** is a hierarchical data structure consisting of nodes connected by edges. It is often used to represent relationships, such as file systems or organizational structures.

- **Node:** A unit of data.
- **Edge:** A connection between two nodes.
- **Root:** The topmost node in a tree. There is only one root node in a tree.
- **Leaf:** A node that does not have any children.
- **Parent:** A node that has one or more child nodes.
- **Child:** A node that is a descendant of another node (parent).
- **Subtree:** A tree formed by a node and all its descendants.
- **Depth of a node:** The number of edges from the root to the node.
- **Height of a tree:** The length of the longest path from the root to a leaf node.
- **Degree of a node:** The number of children of that node.



Types of Trees

1. Binary Tree

1. Each node has at most two children.
2. It can have two types of children: left and right.

2. Binary Search Tree (BST)

1. A type of binary tree where for each node:
 1. The value of the left child is less than the parent node.
 2. The value of the right child is greater than the parent node.
2. This property helps with efficient searching, insertion, and deletion.

3. Balanced Trees

1. **AVL Tree:** A self-balancing binary search tree where the difference between the heights of the left and right subtrees is at most 1 for every node.
2. **Red-Black Tree:** A type of balanced binary search tree with additional properties to maintain balance.



Types of Trees

4. Heap

4. A special type of binary tree used for priority queues.
5. A **max heap** has the property that each parent node has a value greater than or equal to its children.
6. A **min heap** has the property that each parent node has a value less than or equal to its children.

5.B-tree

1. A self-balancing tree data structure used in databases and file systems.
2. It maintains sorted data and allows for efficient insertion, deletion, and search operations.

6.Trie (Prefix Tree)

1. A tree-like data structure used to store strings, where each node represents a character of a string.
2. Commonly used for autocomplete and spell-checking.



Tree Traversal

Tree traversal is the process of visiting all the nodes in a tree in a specific order.

1. Pre-order Traversal

1. Visit the root node first, then traverse the left subtree, then the right subtree.
2. Order: Root $\rightarrow$ Left $\rightarrow$ Right

2. In-order Traversal (used in BST)

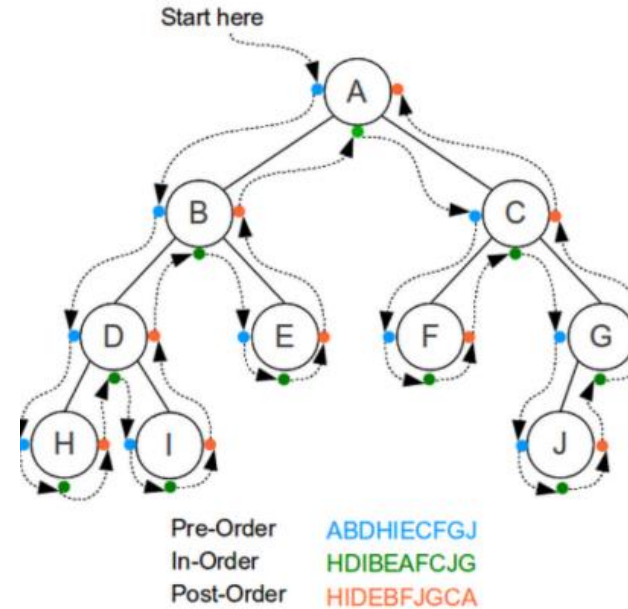
1. Traverse the left subtree, visit the root node, and then traverse the right subtree.
2. Order: Left $\rightarrow$ Root $\rightarrow$ Right
3. In-order traversal of a BST gives the nodes in ascending order.

3. Post-order Traversal

1. Traverse the left subtree first, then the right subtree, and then visit the root node.
2. Order: Left $\rightarrow$ Right $\rightarrow$ Root

4. Level-order Traversal (Breadth-First Traversal)

1. Visit all the nodes level by level, starting from the root and then moving to the next level.
2. This is often implemented using a queue.



Pre-Order: Root-Left-Right
In-Order: Left-Root-Right
Post-Order: Left-Right-Root



Binary Search Tree (BST) Operations

A **Binary Search Tree (BST)** is a tree data structure where each node has at most two children, and for each node, the value of its left child is smaller and the value of its right child is greater.

Basic Operations

1. Insertion

1. Insert a new node into the BST by following the rule:
 1. If the new value is less than the current node, go to the left child.
 2. If the new value is greater, go to the right child.

2. Search

1. To search for a value, start at the root and follow the left or right child depending on whether the target value is smaller or larger than the current node.

3. Deletion

1. There are three cases to handle
 1. **Node with no children (leaf node):** Simply remove the node.
 2. **Node with one child:** Replace the node with its child.
 3. **Node with two children:** Find the in-order successor (smallest node in the right subtree) or in-order predecessor (largest node in the left subtree), replace the node with it, and then remove the successor or predecessor.

4. Traversal

1. Perform any of the standard traversals (Pre-order, In-order, Post-order).



Graph Algorithms

กราฟคืออะไร?

กราฟ (Graph) เป็นโครงสร้างข้อมูลประกอบด้วย จุด (Nodes/Vertices) และ เส้นเชื่อม (Edges) ที่ใช้เชื่อมโยงจุดต่างๆ เข้าด้วยกัน โดยสามารถใช้ในการแก้ปัญหามากมายประเภท เช่น

- ✓ การหาเส้นทางที่สั้นที่สุด
- ✓ การค้นหาข้อมูลที่เชื่อมโยงกัน
- ✓ การวิเคราะห์เครือข่ายทางสังคม
- ✓ การวางแผนเส้นทางเดินรถ



โครงสร้างของกราฟ

กราฟสามารถแบ่งออกเป็น 2 ประเภทหลัก

1. กราฟมีทิศทาง (Directed Graph - Digraph)

1. มีทิศทางของเส้นเชื่อม ($A \rightarrow B$ ไม่เหมือน $B \rightarrow A$)
2. ตัวอย่าง ระบบถนนแบบวันเวย์, กราฟแสดงการติดตามบนโซเชียลมีเดีย

2. กราฟไม่มีทิศทาง (Undirected Graph)

1. เส้นเชื่อมไม่มีทิศทาง ($A \leftrightarrow B$ เท่ากับ $B \leftrightarrow A$)
2. ตัวอย่าง กราฟแสดงความสัมพันธ์เพื่อนบน Facebook



การแทนกราฟในโปรแกรม

สามารถแทนกราฟได้หลายวิธี เช่น

1. Adjacency Matrix (เมทริกซ์ของการเชื่อมโยง)

1. ใช้ **2D Array** เก็บค่าความสัมพันธ์ระหว่างจุด
2. เหมาะสำหรับกราฟที่มีการเชื่อมต่อเยอะ
3. ข้อเสียใช้หน่วยความจำเยอะ

2. Adjacency List (ลิสต์ของการเชื่อมโยง)

1. ใช้ **List หรือ Vector** เก็บเฉพาะจุดที่มีการเชื่อมต่อ
2. เหมาะสำหรับกราฟที่มีจุดจำนวนมากแต่เส้นเชื่อมน้อย
3. ข้อดีใช้หน่วยความจำน้อยกว่า



อัลกอริทึมที่ใช้กับกราฟ

การค้นหาในกราฟ

1. BFS (Breadth-First Search) – การค้นหาแบบกว้างก่อน

1. ใช้ **Queue** เพื่อไล่สำรวจระดับต่อระดับ
2. เหมาะกับการหาเส้นทางที่สั้นที่สุดในกราฟที่ไม่มีน้ำหนัก

2. DFS (Depth-First Search) – การค้นหาแบบลึกก่อน

1. ใช้ **Stack** (หรือ Recursion) ในการเดินสำรวจ
2. เหมาะกับการตรวจจับวงจร หรือการหาทางเชื่อมโยงข้อมูล



การหาเส้นทางที่สั้นที่สุด

1. Dijkstra's Algorithm

1. ใช้กับกราฟที่มี ค่าน้ำหนักเป็นบวก
2. เหมาะสำหรับปัญหาหาเส้นทางที่สั้นที่สุด เช่น GPS

2. Bellman-Ford Algorithm

1. ใช้กับกราฟที่มี น้ำหนักติดลบได้
2. ใช้ได้ในกรณีที่ Dijkstra ใช้ไม่ได้

3. Floyd-Warshall Algorithm

1. ใช้หาทางสั้นที่สุดระหว่าง ทุกคู่ของจุด
2. ใช้ Dynamic Programming



Brute Force

Brute Force

- Brute Force เป็นวิธีการแก้ปัญหาที่ง่ายที่สุด เพราะไม่จำเป็นต้องคิดวิธีที่ดีอะไรมา เพียงแค่ทดลองทุกคำตอบ/ทุกกรณีที่เป็นไปได้ แล้วตอบคำตอบที่ดีที่สุด
 - หรือตอบจำนวนวิธีที่เป็นไปได้ หากโจทย์ถามหาจำนวนวิธีในการทำอะไรบางอย่าง
- โจทย์ส่วนใหญ่จะให้คะแนนวิธีนี้น้อย แต่ควรศึกษาและฝึก implement Brute Force เพราะหากหาวิธีที่ดีไม่ได้ การทำ Brute Force ก็อาจจะช่วยได้บางส่วน

Iterative Complete Search

- โจทย์บางข้อสามารถ Brute Force ได้โดยการทดลองรูปแบบคำตอบทั้งหมดที่เป็นไปได้ โดยอาจจะลูบหลายชั้น เช่น
- กำหนดลำดับตัวเลขมาให้ n ตัว ให้เลือกตัวเลขสองตัวที่มีค่าต่างกันน้อยที่สุดที่เป็นไปได้
 - วิธีหนึ่งที่ทำได้ คือลูบสองชั้นเพื่อทดลองเลือกตัวเลขสองตัวทุกแบบที่เป็นไปได้ หาวิธีการเลือกที่ได้ค่าแตกต่างกันน้อยที่สุดแล้วตอบ
- กำหนดจำนวนเต็ม A และ B ให้ จงหาจำนวนเต็มบวก x และ y ทุกคู่ที่ตรงตามเงื่อนไข $x+y = A$ และ $x*y = B$
 - ใช้การทดลองลูบสองชั้นเพื่อทดลองค่า x, y ทั้งหมดที่เป็นไปได้ (ไม่เกิน A หรือ B) แล้วตรวจสอบเงื่อนไขทั้งสองเงื่อนไข ถ้าตรงเงื่อนไขก็ตอบคู่ นั้น วิธีนี้จะใช้เวลา $O(\min(A, B)^2)$
 - สังเกตว่าความจริงแล้วเราไม่จำเป็นต้องลูบค่า y ก็ได้ เพราะหากเลือกค่า x แล้ว เราสามารถสรุปได้ทันทีว่า $y = A-x$ ตามสมการแรก สิ่งที่ต้องทำมีเพียงแค่การตรวจสอบเงื่อนไข $x*y = B$ ว่าเป็นจริงหรือไม่ วิธีนี้จะใช้เวลา $O(\min(A, B))$

Recursive Complete Search

โจทย์บางข้อจำเป็นต้องทดลองวิธีเป็นจำนวนมาก และไม่สามารถใช้รูปหลายชั้นในการแก้ได้ ดังนั้นจึงต้องเขียน recursive function เพื่อทดลองเลือกทุกวิธีที่เป็นไปได้แทน

Recursive Complete Search

- ต้องการทำอาหาร มีส่วนผสมให้ทั้งหมด n ชิ้น ($n \leq 10$) โดยส่วนผสมแต่ละชิ้นจะมีค่าความเปรี้ยวและค่าความขม ให้เลือกส่วนผสมมาทำอาหาร (เลือกกี่ชิ้นก็ได้ แต่ต้องอย่างน้อย 1 ชิ้น) เมื่อทำอาหาร จะได้อาหารที่มีค่าความเปรี้ยวเท่ากับผลคูณของค่าความเปรี้ยวของส่วนผสมที่เลือกมา ส่วนค่าความขมจะเท่ากับผลบวกของค่าความขมของส่วนผสมที่เลือกมา
- ให้นำวิธีเลือกโดยให้ความต่างระหว่างค่าความขมกับค่าความเปรี้ยวน้อยที่สุดเท่าที่เป็นไปได้



Recursive Complete Search

```
#include <bits/stdc++.h>
using namespace std;
const int N = 20;
const int INF = 1e9;
int n;
int sour[N], bitter[N];
bool picked[N];
int ans = INF;

int main()
{
    scanf("%d", &n);
    for (int i = 1; i <= n; ++i)
        scanf("%d%d", &sour[i], &bitter[i]);
    pick(1);
    printf("%d\n", ans);

    return 0;
}

void pick(int i)
{
    if (i <= n) {
        picked[i] = true;
        pick(i+1);
        picked[i] = false;
        pick(i+1);
    } else { // i == n+1
        int total_sour = 1, total_bitter = 0, count_used = 0;
        for (int j = 1; j <= n; ++j) {
            if (picked[j]) {
                ++count_used;
                total_sour *= sour[j];
                total_bitter += bitter[j];
            }
        }
        if (count_used != 0)
            ans = min(ans, abs(total_sour - total_bitter));
    }
}
```



Divide and Conquer

Divide and Conquer

- แนวคิดหลัก ๆ ของ Divide and Conquer คือ แบ่งปัญหาออกเป็นหลายส่วนที่เท่า ๆ กัน ใช้ อัลกอริทึมเดิมแก้ปัญหาย่อย ๆ ซ้ำแบบ recursive แล้วหาวิธีรวมคำตอบเข้าด้วยกัน เป็น คำตอบของปัญหาใหญ่ (โดยส่วนใหญ่ จะแบ่งออกเป็น 2 ส่วนเท่า ๆ กัน) เช่น
 - Merge Sort ที่ใช้การ sort array ครั้งแรกกับครึ่งหลังแยกกัน แล้วค่อยนำ array ทั้งสอง มารวมกัน เนื่องจากว่ามีการแบ่งครึ่งลงไปทั้งหมด $O(\log n)$ ชั้น แต่ละชั้นจะต้องจัดการกับ สมาชิกของ array เพื่อทำการรวม array - รวมทั้งหมด $O(n)$ ตัว ทำให้เวลาการทำงานของ merge sort เป็น $O(n \log n)$
 - Binary Search บนปัญหา Optimization Problem ที่ถูกแปลงเป็น Decision Problem ซึ่งพบได้บ่อยมาก ควรศึกษาไว้

Binary Search

- กำหนดลำดับจำนวนให้ทั้งหมด n ตัว มีคำถามให้ q คำถาม แต่ละคำถาม ให้หาว่ามีเลข x อยู่ในลำดับหรือไม่
 - เราสามารถ sort array ดังกล่าวเรียงจากน้อยไปมากก่อน แล้วสำหรับแต่ละคำถาม เราจะมีวิธีดังนี้
 - ดูตำแหน่งกลางของ array ก่อน หากตัวเลขตรงกับตัวที่ต้องการหา ก็จบการทำงาน ตอบได้ว่ามีตัวเลขดังกล่าว
 - หากไม่ใช่ ให้ดูว่าตัวเลขที่เราต้องการมีค่าน้อยกว่าหรือมากกว่าตัวเลขตำแหน่งตรงกลาง
 - หากตัวเลขที่เราต้องการมีค่าน้อยกว่า แสดงว่าอาจจะอยู่ทางซ้าย เราจะมาค้นหาในช่วงครึ่งซ้ายแทน แต่หากมากกว่า ให้ค้นหาในช่วงครึ่งขวา
 - การค้นหาแต่ละช่วง จะใช้ดูค่าตรงกลาง แล้วเลือกครึ่งที่สมควรค้นหาเรื่อย ๆ เช่นเดียวกัน
 - แต่ละครั้ง array ของเราจะถูกตัดออกไปทีละครึ่ง ดังนั้น รวมแล้วจะใช้เวลาไม่เกิน $O(\log n)$ ในการตอบคำถามแต่ละครั้ง



การแปลง Optimization Problem เป็น Decision Problem

- ปัญหาที่ต้องการให้หาค่าน้อยสุด/มากที่สุดบางอย่าง สามารถแปลงมาเป็นปัญหาการตัดสินใจ (ที่ต้องตอบ Yes/No) เพื่อที่จะทำให้หาคำตอบโดยการ binary search ได้ ทำให้แก้ปัญหได้ง่ายขึ้น ยกตัวอย่างดังนี้

การแปลง Optimization Problem เป็น Decision Problem

- มีลำดับจำนวนเต็มบวกอยู่ n ตัว ต้องการแบ่งเป็นช่วง ๆ โดยจะมีค่า x กำหนดไว้ แต่ละช่วงจะต้องมีผลรวมของสมาชิกไม่เกิน x ถามว่า หากเราต้องการแบ่งออกเป็นช่วงไม่เกิน k ช่วง เราสามารถกำหนดค่า x ให้น้อยสุดได้เท่าไร
- ตัวอย่าง 3, 5, 1, 2, 4 และ $k = 2$
 - หากเรากำหนดให้ $x = 7$ จะพบว่า การแบ่งโดยให้แต่ละช่วงมีผลรวมไม่เกิน $x = 7$ จะต้องใช้อย่างน้อย 3 ช่วง เช่น [3], [5], [1, 2, 4] - ค่า x นี้ใช้ไม่ได้ เพราะเราต้องการแบ่งเป็นช่วงไม่เกิน $k = 2$ ช่วง
 - หากเรากำหนดให้ $x = 8$ พบว่าเราสามารถแบ่งเป็น [3, 5], [1, 2, 4] ซึ่งใช้เพียง $k = 2$ ช่วงเท่านั้น ไม่ขัดกับเงื่อนไขโจทย์
- หากเราเปลี่ยนปัญหาจาก "หาค่า x ที่น้อยสุดที่เป็นไปได้" เป็น "ตรวจสอบว่า ถ้าแบ่งให้แต่ละช่วงมีผลรวมไม่เกิน x จะแบ่งให้มีไม่เกิน k ช่วงได้หรือไม่" เราจะตอบปัญหาแบบหลังได้ง่ายขึ้น

การแปลง Optimization Problem เป็น Decision Problem

- การตรวจสอบว่าทำได้หรือไม่ สามารถทำได้แบบ greedy
 - เริ่มต้นมา ให้ช่วงแรกประกอบด้วยจำนวนตัวแรกเพียงตัวเดียว
 - พยายามเพิ่มตัวถัดไปเรื่อย ๆ トラバิดที่ผลรวมยังไม่เกิน x
 - ถ้าเกินแล้ว ให้เริ่มต้นช่วงใหม่
 - ทำไปเรื่อย ๆ จนครบทุกตัว แล้วตรวจสอบว่าจำนวนช่วงเกิน k หรือไม่
- เมื่อเราสามารถตรวจสอบคำตอบได้อย่างง่ายดาย การที่จะแก้ปัญหาดังเดิมได้นั้น ก็เพียงแค่ binary search บนค่า x โดยมีเงื่อนไขดังนี้
 - เริ่มต้นมา พิจารณาค่า x ที่เป็นไปได้ตั้งแต่ 1 ถึง INF (เมื่อ INF คือ bound ผลรวมที่มากที่สุดที่เป็นไปได้ของ array ทั้ง array)
 - ทดลองค่า x ตรงกลางช่วง หากตรวจสอบแล้วสามารถทำให้เงื่อนไขเป็นจริงได้ สังเกตว่าถ้าเราเพิ่มค่า x ยิ่งเงื่อนไขก็ยิ่งเป็นจริงอยู่ แต่เราต้องการค่า x น้อยสุด ดังนั้น เราจะพิจารณาเฉพาะช่วงคำตอบ x ตั้งแต่ 1 ถึงค่าดังกล่าว (เพื่อให้ x น้อยกว่านี้ได้อีก)
 - หากทดลองแล้วพบว่าเงื่อนไขเป็นเท็จ นั่นแปลว่าเราจำเป็นต้องเพิ่มค่า x เพราะฉะนั้น เราจะพิจารณาค่า x ที่เป็นไปได้ ตั้งแต่ ค่าดังกล่าวบวก 1 ถึง INF
 - เราจะแบ่งครึ่งช่วงไปเรื่อย ๆ จนกว่าจะเหลือเพียงค่าเดียว ได้เป็นคำตอบของคำถามดังกล่าว



การแปลง Optimization Problem เป็น Decision Problem

```
#include <bits/stdc++.h> //  
<- include everything  
using namespace std;
```

```
const int N = 100010;  
const long long INF = 1e18;
```

```
int n, k;  
long long val[N];
```

```
int main()  
{  
    scanf("%d%d", &k, &n);  
    for (int i = 1; i <= n; ++i)  
        scanf("%lld", &val[i]);  
  
    long long lo = 1;  
    long long hi = INF;  
    while (lo < hi) {  
        long long mid = (lo+hi)/2;  
        if (check(mid))  
            hi = mid;  
        else  
            lo = mid+1;  
    }  
    printf("%lld\n", lo);  
  
    return 0;  
}
```

```
bool check(long long limit)  
{  
    int cnt = 1;  
    long long sum = 0;  
    for (int i = 1; i <= n; ++i) {  
        if (val[i] > limit)  
            return false;  
        // เริ่มต้นช่วงใหม่ ถ้าเพิ่มตัวใหม่เข้า ไปแล้วทำให้ไม่เกิน  
        if (sum + val[i] > limit) {  
            ++cnt;  
            sum = 0;  
        }  
        sum += val[i];  
    }  
    return cnt <= k;  
}
```



Greedy

Greedy

- Greedy Algorithm คือขั้นตอนวิธีที่ใช้วิธีการเลือกตัวเลือกที่ดีที่สุด เท่าที่เห็น ณ ตอนนั้นโดยไม่คำนึงถึงว่าการเลือกตัวเลือกดังกล่าวจะทำให้คำตอบนั้นดีที่สุดหรือไม่
- โดยปกติจะใช้ Greedy Algorithm ในกรณีที่พิสูจน์ได้ว่า ถึงเลือกตัวเลือกไปแบบไม่คิดอะไรมาก ณ ตอนนั้น ทำเช่นนี้ไปเรื่อย ๆ ย่อมได้คำตอบที่ดีที่สุดเสมอ
- ตัวอย่าง Greedy Algorithm อย่าง Dijkstra's Algorithm, Kruskal's Algorithm, Prim's Algorithm



Backtracking



Problem Solving with Backtracking

Backtracking is a problem-solving technique used to find solutions to problems that require exploring all possible configurations and systematically eliminating the ones that do not meet the criteria.

It's particularly effective for problems related to combinatorics, constraint satisfaction, and optimization, where a solution involves making a series of choices. Here's an overview of **how backtracking works**, followed by several **examples of problems solved with backtracking**

How Backtracking Works

- Backtracking is a recursive algorithm that tries to build a solution incrementally.
 - If at any point, the partial solution doesn't seem to lead to a valid solution, it backtracks to the previous step and tries a different approach.
1. Start with an empty solution and gradually build the solution piece by piece.
 2. Make a choice (usually a decision or an action that moves toward the solution).
 3. Check for validity of the current partial solution.
 4. If valid, recursively proceed by exploring the next level or choice.
 5. If invalid, backtrack by undoing the last choice and trying a different one.
 6. Repeat the process until a solution is found, or all possibilities are exhausted.

The algorithm typically follows a Depth-First Search (DFS) approach to explore all potential solutions.



Backtracking Example- Subset Sum Problem

Problem: Given a set of integers, find all subsets that sum to a target value.

Solution using Backtracking

1. Start with an empty subset.
2. Choose an element from the set and add it to the current subset.
3. Check if the sum of the current subset matches the target value.

 If it does, save the subset as a solution.

 If not, continue the process by either adding more elements or backtracking by removing the last element.

4. Backtrack if the current subset exceeds the target sum or if no solution can be found in the current branch.

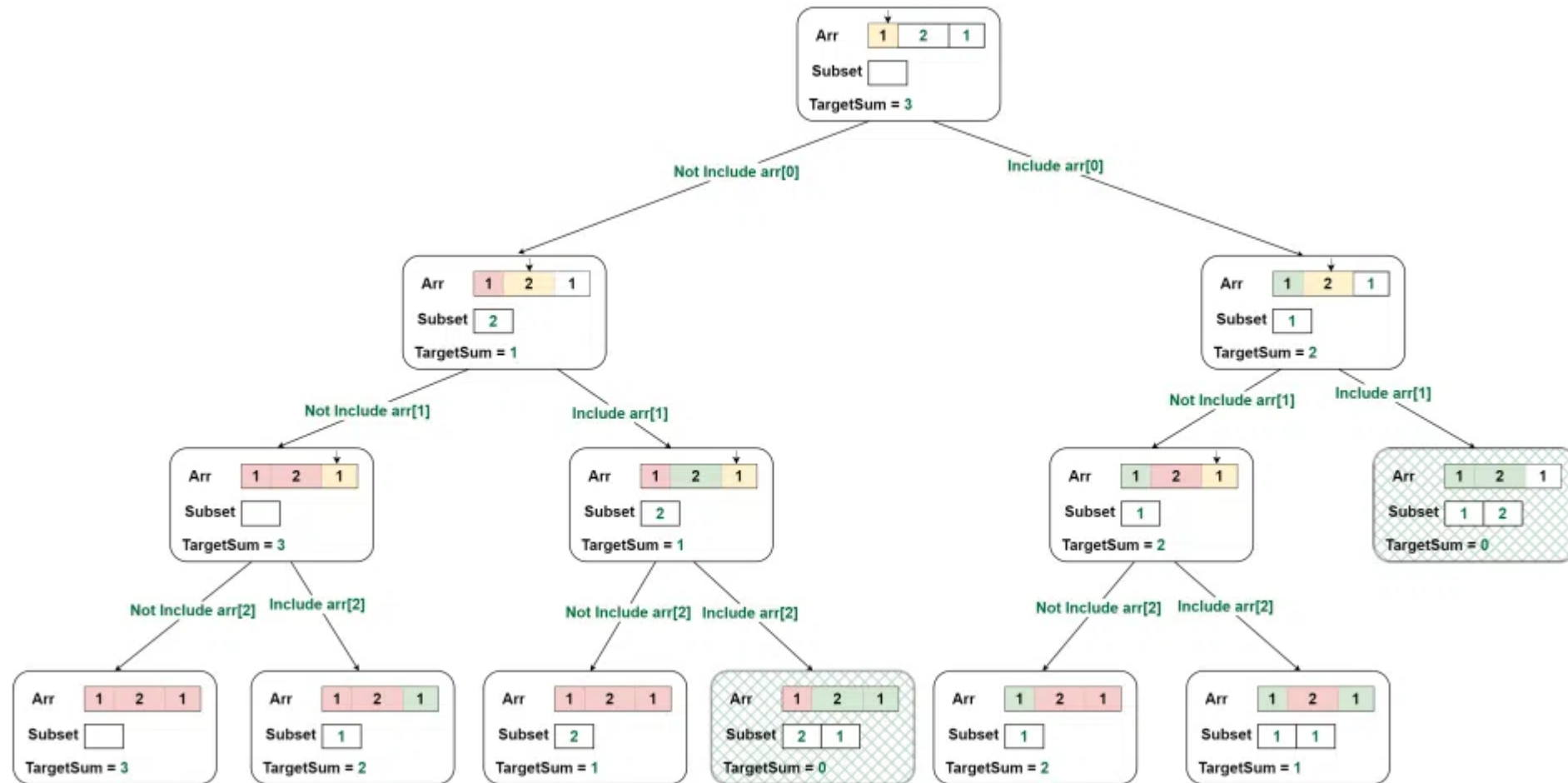


Backtracking Example- Subset Sum Problem

Backtracking Steps

1. Choose each element one by one and include it in the subset.
2. After adding an element, check if the subset sum equals the target.
3. If the sum exceeds the target, backtrack and try a different element.
4. Continue exploring all possible combinations.

Backtracking Example- Subset Sum Problem



Subset Sum Problem



Backtracking Example- Subset Sum Problem

```
#include <bits/stdc++.h>
using namespace std;
bool flag = 0;
void PrintSubsetSum(int i, int n, int set[], int targetSum, vector<int>& subset)
{
    // targetSum is zero then there exist a subset.
    if (targetSum == 0) {
        // Prints valid subset
        flag = 1;
        cout << "[ ";
        for (int i = 0; i < subset.size(); i++) {
            cout << subset[i] << " ";
        }
        cout << "]\n";
        return;
    }
    if (i == n) {
        // return if we have reached at the end of the array
        return;
    }
    // Not considering current element
    PrintSubsetSum(i + 1, n, set, targetSum, subset);
    // consider current element if it is less than or equal
    // to targetSum
    if (set[i] <= targetSum) {
        // push the current element in subset
        subset.push_back(set[i]);

        // Recursive call for consider current element
        PrintSubsetSum(i + 1, n, set, targetSum - set[i],
                        subset);

        // pop-back element after recursive call to restore
        // subsets original configuration
        subset.pop_back();
    }
}
```

```
int main()
{
    // Test case 1
    int set[] = { 1, 2, 1 };
    int sum = 3;
    int n = sizeof(set) / sizeof(set[0]);
    vector<int> subset;
    cout << "Output 1:" << endl;
    PrintSubsetSum(0, n, set, sum, subset);
    cout << endl;
    flag = 0;
    // Test case 2
    int set2[] = { 3, 34, 4, 12, 5, 2 };
    int sum2 = 30;
    int n2 = sizeof(set) / sizeof(set[0]);
    vector<int> subset2;
    cout << "Output 2:" << endl;
    PrintSubsetSum(0, n2, set2, sum2, subset2);
    if (!flag) {
        cout << "There is no such subset";
    }

    return 0;
}
```



Backtracking Example- Permutation Generation

Problem

Generate all possible permutations of a given set of distinct numbers.

Solution using Backtracking

1. Start with an empty permutation.
2. Recursively build the permutation by adding one number at a time.
3. After each addition, check if the permutation is complete (i.e., it has the same number of elements as the original set).
4. If valid, add the permutation to the list of results.
5. Backtrack by removing the last number added and trying the next available number.

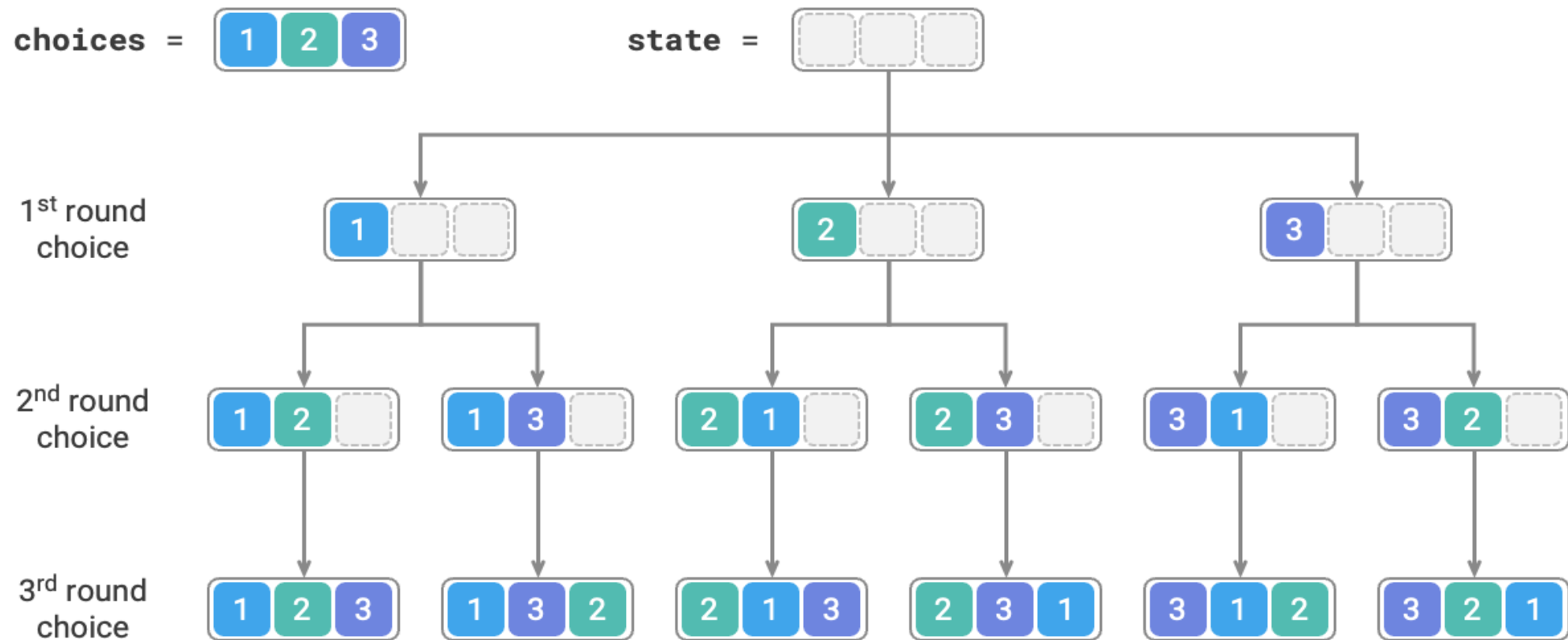


Backtracking Example- Permutation Generation

Backtracking Steps:

1. Consider all numbers in the set, one by one.
2. Swap elements to generate different permutations.
3. After generating a complete permutation, backtrack by swapping the numbers back and trying the next one.

Backtracking Example- Permutation Generation





Backtracking Example- Permutation Generation

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

// Function to print the permutation
void printPermutation(const vector<int>& perm) {
    for (int num : perm) {
        cout << num << " ";
    }
    cout << endl;
}

// Backtracking function to generate all permutations
void generatePermutations(vector<int>& nums, int start) {
    // If we've reached the end of the array, print the current permutation
    if (start == nums.size()) {
        printPermutation(nums);
        return;
    }
    // Swap each element with the current index and generate permutations
    for (int i = start; i < nums.size(); ++i) {
        // Swap the current element with the start element
        swap(nums[start], nums[i]);
        // Recur to generate permutations with the next element
        generatePermutations(nums, start + 1);
        // Backtrack by swapping the elements back to their original positions
        swap(nums[start], nums[i]);
    }
}

int main() {
    vector<int> nums = {1, 2, 3}; // Set of distinct numbers

    cout << "All permutations are:\n";

    // Call the backtracking function to generate all permutations
    generatePermutations(nums, 0);

    return 0;
}
```



General Steps for Solving Problems with Backtracking

- 1. Define the problem space:** Understand the search space and how decisions are made at each step.
- 2. Recursive Structure:** Identify how the problem can be broken down into smaller subproblems. Each step in recursion should make progress towards the solution.
- 3. Choose an element:** Select the next part of the solution and try it.
- 4. Check constraints:** After each step, verify whether the partial solution is valid.
- 5. Prune:** If the solution cannot be completed from the current step, backtrack by undoing the last decision and trying the next option.
- 6. Termination:** The recursion ends when a valid solution is found, or when all possible configurations have been explored.

ทดลองทำโจทย์